

คู่มือติดตั้งและตั้งค่าระบบบอท Google Chat + Jira + Dashboard

คู่มือการตั้งค่าและการขอ API Key/Token อย่างละเอียดทีละขั้นตอน

VERSION 1.0 (PRODUCTION)

วันที่จัดทำ: 7 กรกฎาคม 2569

จัดทำโดย: **Antigravity AI (Google DeepMind)**

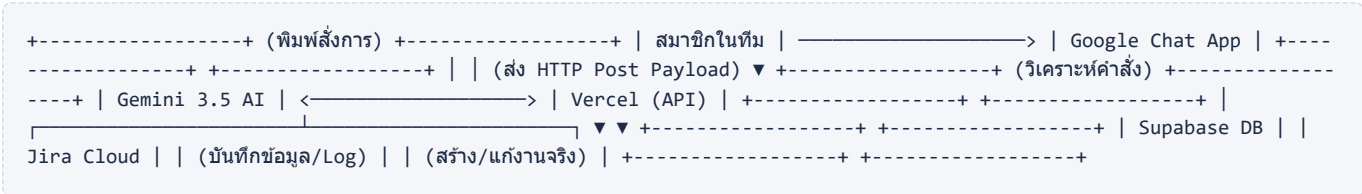
คู่มือฉบับสมบูรณ์จัดทำโดย Antigravity AI (Google DeepMind) → เอกสารสำหรับใช้งานภายในทีมเท่านั้น

สารบัญเอกสาร

1. ภาพรวมการไหลของข้อมูลในระบบ (System Flow)
2. รายการ Environment Variables ทั้งหมดที่ต้องใช้
3. ขั้นตอนการตั้งค่า Supabase → คัดลอก DATABASE_URL
4. ขั้นตอนการตั้งค่า Jira Cloud → คัดลอก JIRA_*
5. ขั้นตอนการขอ Google Gemini API Key
6. การสร้าง Google Chat Webhook URL (สำหรับส่งสรุปรายวัน)
7. การตั้งค่าบอทเพื่อรับคำสั่ง (Google Chat App)
8. การตั้งค่า Jira Webhook เพื่อ Sync ตารางงานกลับ
9. การเตรียมระบบบนเครื่องคอมพิวเตอร์ของคุณ (Local)
10. การ Deploy ขึ้น Vercel และกรอกค่า Config
11. สรุปลำดับส่งการบอทใน Google Chat
12. วิธีแก้ไขปัญหาที่พบบ่อย (Troubleshooting)

1. ภาพรวมการไหลของข้อมูลในระบบ (System Flow)

ระบบนี้ถูกออกแบบมาเพื่อลดความยุ่งยากในการประสานงานระหว่างทีมกับบอร์ดโปรเจกต์ Jira โดยใช้ AI เข้ามาช่วยแปลงภาษาพูดธรรมดาให้เป็นคำสั่งจัดการตัวงานในระบบอัตโนมัติ ซึ่งสถาปัตยกรรมระบบมีขั้นตอนดังต่อไปนี้:



💡 ข้อมูลการทำงานของระบบย่อย:

ระบบจะทำงานร่วมกันสามส่วนคือ Dashboard บน Vercel ทำหน้าที่เป็นสมองกลางคอยเรียก Gemini AI เพื่อตีความคำสั่ง เมื่อ AI แปลงคำสั่งเสร็จสิ้น ระบบจะยิง API ไปยัง Jira เพื่อปรับปรุงบอร์ดงาน แล้วบันทึกความเคลื่อนไหวลงฐานข้อมูล Supabase เพื่อให้หน้า Dashboard แสดงข้อมูลได้ตรงกับความเป็นจริงอยู่เสมอ

2. รายการ Environment Variables

ด้านล่างนี้คือรายการตัวแปรสภาพแวดล้อม (Environment Variables) ทั้งหมดที่ต้องกรอกในไฟล์ `.env` ของเครื่อง
เครื่องผู้พัฒนา และบนระบบคลาวด์ Vercel สำหรับใช้งานในโหมด Production:

ชื่อตัวแปร (Key)	ตัวอย่างค่า (Value)	ความสำคัญ
DATABASE_URL	postgresql://postgres.xxx:xxx@aws.pooler.supabase.com:6543/postgres	จำเป็นมาก (จำเป็นต้องใช้พอร์ต 6543)
JIRA_DOMAIN	yourcompany.atlassian.net	จำเป็นมาก (ห้ามใส่ https://)
JIRA_EMAIL	bot-account@company.com	จำเป็นมาก (อีเมลเจ้าของ Token)
JIRA_API_TOKEN	ATATT3xFfGF0tVuCD...	จำเป็นมาก (สิทธิ์แอดมินหรือผู้สร้างงาน)
JIRA_PROJECT_KEY	TES (หรือคีย์โครงการอื่นๆ)	จำเป็นมาก (ตัวพิมพ์ใหญ่ทั้งหมด)
GEMINI_API_KEY	AIzaSy... (ต้องขึ้นต้นด้วย AIzaSy เสมอ)	จำเป็นมาก (ใช้เชื่อมกับโมเดล Gemini)
GEMINI_MODEL	gemini-3.5-flash	แนะนำ (โมเดลใหม่และเร็วที่สุด)
GOOGLE_CHAT_WEBHOOK_URL	https://chat.googleapis.com/v1/spaces/...	จำเป็น (ใช้ยิงรายงานสรุปรายวัน)

ชื่อตัวแปร (Key)	ตัวอย่างค่า (Value)	ความสำคัญ
BOT_NAME	sekloso	แนะนำ (ชื่อบอทแสดงในแชท)
SESSION_SECRET	สุ่มรหัสความยาว 64 ตัวอักษร	จำเป็นมาก (เพื่อความปลอดภัยของระบบเว็บ)
CRON_SECRET	สุ่มรหัสความยาว 32 ตัวอักษร	จำเป็นมาก (เพื่อความปลอดภัยของระบบแจ้งเตือน)

 **กฎเหล็กด้านความปลอดภัย:**

ห้ามนำไฟล์ `.env` หรือโค้ดที่มีค่า Key ส่วนตัวด้านบนนี้อัปโหลด (Push) ขึ้นสู่ GitHub สาธารณะเด็ดขาด เพราะหากรั่วไหลออกไป ผู้อื่นจะสามารถควบคุมระบบบอร์ດงานของบริษัทและลบข้อมูลทำงานของคุณได้ทันที!

3. ขั้นตอนการตั้งค่า Supabase → คัดลอก DATABASE_URL

Supabase ทำหน้าที่เก็บข้อมูลรายชื่อทีม แคมเปญงาน และล็อกกิจกรรมเพื่อใช้ในการเปิดหน้าเว็บแผงควบคุม (Dashboard):

ขั้นตอนการทำงาน (อิงตาม UI ของ Supabase เวอร์ชันล่าสุด):

1. เข้าไปที่เว็บ [Supabase.com](https://supabase.com) จากนั้นเข้าสู่ระบบด้วยบัญชีของคุณ
2. คลิกปุ่ม **New Project** ตั้งชื่อ เช่น `jira-bot-production`
3. ตั้งรหัสผ่านสำหรับฐานข้อมูล (Database Password) และจดรหัสผ่านนี้ไว้เพื่อนำมาเปลี่ยนลงใน URL
4. เลือกเซิร์ฟเวอร์ในภูมิภาค **Singapore (ap-southeast-1)** เพื่อประสิทธิภาพสูงสุด
5. รอจนระบบสร้างเซิร์ฟเวอร์เสร็จสิ้น (ใช้เวลาประมาณ 1-2 นาที)
6. เมื่อพร้อมใช้งาน ให้กดปุ่ม **Connect** (มุมขวามือของหน้าเว็บ)
7. ในหน้าต่างที่ปรากฏขึ้น ให้เลือกแท็บเมนู **ORM** (Third-party library) ด้านบนสุด
8. ในช่อง **Connection Method** ให้คลิกเลือก **Transaction pooler**
9. ในช่อง **Type** ให้ตรวจสอบว่าเลือกเป็น **URI**
10. คัดลอก Connection String ที่ปรากฏในกล่องข้อความด้านล่าง ซึ่งจะมีโครงสร้างดังนี้:

```
postgresql://postgres:aoahyspiorxdfyloekfo:[YOUR-PASSWORD]@aws-1-ap-southeast-1.pooler.supabase.com:6543/postgres
```

11. แทนที่ข้อความ `[YOUR-PASSWORD]` ด้วยรหัสผ่านจริงที่คุณตั้งไว้ในขั้นตอนที่ 3 และบันทึกค่าเก็บไว้

💡 ทำไมต้องเลือก ORM → Transaction pooler?

เนื่องจากสถาปัตยกรรมของบอทที่รันอยู่บน Next.js Serverless ซึ่งจะทำให้การเปิดและปิด request สั้นๆ จำนวนมาก โหมด Transaction pooler ของ Supabase (พอร์ต 6543) จะช่วยจัดการการเชื่อมต่อได้ดีที่สุดและไม่ทำให้ Database Connection เกินขีดจำกัดของแผนการใช้งานฟรี

4. ขั้นตอนการตั้งค่า Jira Cloud → คัดลอก JIRA_*

ในการทำให้ระบบโปรแกรมมิ่งสามารถสร้างหรืออัปเดตงานบน Jira Cloud ได้สำเร็จ จะต้องระบุค่าดังต่อไปนี้:

4.1 โดเมนหลัก (JIRA_DOMAIN)

คัดลอกเฉพาะที่อยู่โดเมนของคุณจาก URL หน้าบอร์ด Jira โดย **ไม่ต้องใส่** `https://` หรือเครื่องหมาย Slash ต่อท้ายใดๆ ตัวอย่างเช่น:

```
JIRA_DOMAIN="company-team.atlassian.net"
```

คู่มือฉบับสมบูรณ์จัดทำโดย Antigravity AI (Google DeepMind) → เอกสารสำหรับใช้งานภายในทีมเท่านั้น

4.2 การสร้าง API Token เพื่อใช้เขียนงาน (JIRA_API_TOKEN)

1. เข้าไปที่เว็บไซต์ [Atlassian Manage API Tokens](#) โดยใช้บัญชีที่มีสิทธิ์จัดการบอร์ด
2. คลิกปุ่ม **Create API token**
3. พิมพ์ชื่อระบุตัวตน เช่น `sekloso-chatbot-prod`
4. กดปุ่มสร้าง จากนั้นคลิก **Copy** รหัสโทเคนที่ได้ แล้วบันทึกทันที เนื่องจากรหัสนี้จะแสดงขึ้นมาเพียงครั้งเดียวเท่านั้นเพื่อความปลอดภัย

5. ขั้นตอนการขอ Google Gemini API Key

Gemini AI ทำหน้าที่เป็นสมองกลในการวิเคราะห์ความตั้งใจของผู้ใช้ (Intent) ว่าต้องการสร้างงานย่อย (Task) หรือปิดงาน (Transition) จากภาษาพูดทั่วไป:

ขั้นตอนการทำงาน:

1. เปิดเว็บไซต์ [Google AI Studio](#)
2. ลงชื่อเข้าใช้ด้วยบัญชี Google (แนะนำให้เลือกประเทศเป็น Thailand เพื่อรองรับการเปิด API ในภูมิภาคนี้)
3. ไปที่เมนู **Get API key** บนแถบเครื่องมือด้านซ้าย
4. คลิกปุ่ม **Create API key** → เลือกสร้าง API Key ในโปรเจกต์ใหม่ (Create API key in new project)
5. เมื่อระบบสร้างคีย์เสร็จสิ้น ให้คัดลอกรหัสทั้งหมดเก็บไว้

🔔 ข้อสังเกตสำคัญในการเช็ค API Key:

รหัส Key ของ Google Gemini API สำหรับระบบบอท ****ต้องขึ้นต้นด้วยอักษร AIzaSy...** หรือขึ้นต้นตามระบบความปลอดภัย Google AI Studio ปัจจุบัน**

หากรหัสที่คุณคัดลอกมาขึ้นต้นด้วย `AQ.Ab8RN...` นั่นคือ ****Jira API Token**** อย่าสับสนนำมาใส่สลับช่องเด็ดขาด เพราะจะทำให้ระบบบอทเกิดอาการค้างและไม่ตอบสนองเนื่องจากการทำงานของ API ดิกลับ!

6. การสร้าง Google Chat Webhook URL (แจ้งเตือน/สรุปรายวัน)

ใช้ในการตั้งค่าส่งแจ้งเตือนและสรุปสรุปการทำงานของทีมแบบรายวันอัตโนมัติเมื่อถึงเวลา 06:00 น. ตรง:

ขั้นตอนการทำงาน:

1. เปิดหน้าโปรแกรม Google Chat → เข้าไปในห้องกลุ่มแชทที่ต้องการใช้สำหรับแจ้งเตือน
2. คลิกที่หัวข้อห้องแชทด้านบน → เลือกเมนู **Apps & integrations** (แอปและการผสานรวม)
3. คลิก **Add Webhooks** (เพิ่มเว็บฮุก)
4. กรอกรายละเอียดชื่อ เช่น `Jira Daily Notification` และระบุรูปภาพประกอบหากต้องการ จากนั้นกดบันทึก
5. คัดลอกที่อยู่ลิงก์ Webhook URL ที่ได้มาใช้งาน

7. การตั้งค่าบอทเพื่อรับคำสั่ง (Google Chat App)

การตั้งค่าเพื่อให้สมาชิกทีมพิมพ์สั่งงานบอทได้ด้วยการพิมพ์ `@sekloso` ในห้องแชทของ Google:

ขั้นตอนการทำงาน:

1. เข้าไปที่ [Google Cloud Console](#) และเลือกโปรเจกต์ที่ต้องการใช้งาน
2. ค้นหาบริการที่ชื่อว่า **Google Chat API** ในแถบค้นหาด้านบน จากนั้นคลิกเปิดใช้งาน (Enable)
3. ไปที่แถบเมนู **Configuration** (การกำหนดค่า) และระบุค่าดังต่อไปนี้:
 - **App name:** ชื่อบอทที่จะให้แสดง เช่น `sekloso`
 - **Interactive features:** ดึงเลือก `Receive 1:1 messages` และ `Join spaces and group conversations` เพื่อให้บอทคุยส่วนตัวและอยู่ในกลุ่มแชทได้
 - **Connection settings:** เลือกรูปแบบการเชื่อมต่อแบบ ****App URL**** จากนั้นกรอกลิงก์รับข้อมูลของ Vercel ปลายทาง:

```
https://your-app-domain.vercel.app/api/webhook
```

4. ในส่วน ****Visibility**** เลือกการเปิดเผยบอทเฉพาะภายในองค์กรของคุณ
5. กดบันทึก (Save) เป็นอันเสร็จสิ้นขั้นตอน

8. การตั้งค่า Jira Webhook เพื่อ Sync ตารางงานกลับ

ใช้เพื่อรับสัญญาณการอัปเดต เช่น เมื่อมีสมาชิกในทีมย้ายการดำเนินงานบนหน้าเว็บ Jira โดยตรง แล้วจะให้อัปเดตสถานะกลับมาที่ Dashboard อัตโนมัติ:

ขั้นตอนการทำงาน:

1. เข้าสู่ระบบหน้า Jira Cloud ด้วยบัญชีสิทธิ์แอดมิน
2. ไปที่เมนู **Settings (รูปเฟือง) → System → Webhooks** (อยู่ใต้หมวดหมู่การทำงานด้านล่างสุด)
3. คลิกปุ่ม **Create a WebHook**
4. ตั้งชื่อเว็บฮุก เช่น `Sync to Dashboard Web` และระบุสถานะเป็น Active
5. ในส่วนของ URL ให้ระบุที่อยู่ endpoint ปลายทางของคุณ:

```
https://your-app-domain.vercel.app/api/jira/webhook
```

6. ในส่วน JQL ค้นหาให้ป้อน: `project = YOUR\_PROJECT\_KEY` (เช่น `project = TES`)
7. ในส่วน ****Issue events**** ให้เลือกการดักจับเหตุการณ์:
 - **created** (เมื่อตัวถูกสร้างใหม่)
 - **updated** (เมื่อมีการเปลี่ยนแปลงข้อมูลหรือเปลี่ยนสถานะ)

คู่มือฉบับสมบูรณ์จัดทำโดย Antigravity AI (Google DeepMind) เอกสารสำหรับใช้งานภายในทีมเท่านั้น

- **deleted** (เมื่อมีการลบตัวออก)

8. เลื่อนลงด้านล่างสุดแล้วกด **Save**

9. การเตรียมระบบบนเครื่องคอมพิวเตอร์ของคุณ (Local)

ก่อนนำระบบทั้งหมดขึ้นทำงานบนอินเทอร์เน็ตจริง ควรทดสอบระบบบนเครื่องส่วนตัวให้ถูกต้องก่อนดังนี้:

การทำงานผ่าน Terminal ในโฟลเดอร์โปรเจกต์:

1. เปิดโฟลเดอร์โปรเจกต์ `nextjs-app` ใน Command Prompt หรือ PowerShell
2. รันคำสั่งเพื่อช่วยตั้งค่าระบบ:

```
npm run setup
```

3. ทำตามขั้นตอนตัวช่วยกรอกค่าบนหน้าจอ และกด Enter เพื่อบันทึกค่า
4. เมื่อตัวช่วยจบการทำงาน ระบบจะสุ่มรหัสความปลอดภัยและสร้างไฟล์ `.env` ให้อัตโนมัติ
5. รันจำลองระบบบนเครื่องของคุณด้วยคำสั่ง:

```
npm run dev
```

6. เปิดหน้าเว็บเบราว์เซอร์ไปที่: `http://localhost:3000/api/health` เพื่อเช็คความเชื่อมต่อครบหรือไม่

10. การ Deploy ขึ้น Vercel และกรอกค่า Config

เมื่อพร้อมจะเปิดใช้งานจริง ให้ดำเนินงานตามขั้นตอนเหล่านี้:

ขั้นตอนการทำงาน:

1. Push ไฟล์ทั้งหมดของคุณขึ้นสู่ Repository ของบัญชี **GitHub**
2. ไปที่หน้า Dashboard ของ [Vercel.com](https://vercel.com) แล้วกด Import โปรเจกต์ดังกล่าวเข้ามา
3. ในหน้าตั้งค่าโปรเจกต์ ให้ตรวจสอบว่าได้เลือก **Root Directory** ไปที่โฟลเดอร์ที่มีไฟล์ `package.json` ของโปรเจกต์ Next.js (ในกรณีนี้คือโฟลเดอร์ `nextjs-app`)
4. ไปที่เมนู **Settings** → **Environment Variables** แล้วทำการเพิ่มตัวแปรให้ครบทั้ง 11 ตัวที่ระบุไว้ในบทที่ 2
5. กดปุ่ม **Deploy** และรอจนตัวรันแสดงสถานะเป็นสีเขียวว่า Ready!

🌟 ขั้นตอนสุดท้ายหลัง Deploy สำเร็จ:

หลังจากที่คุณได้ชื่อโดเมนเว็บจริงมาแล้ว เช่น `https://your-domain.vercel.app` → ให้ทำการคัดลอก URL ดังกล่าวกลับไปแก้ไขอัปเดตในช่อง **App URL** ของ **Google Chat API (หัวข้อที่ 7)** และ **Jira Webhook URL (หัวข้อที่ 8)** เพื่อให้ระบบจริงเริ่มทำงานเชื่อมต่อกันโดยสมบูรณ์!

11. สรุปคำสั่งสั่งการบอทใน Google Chat

นี่คือตัวอย่างคำสั่งที่ใช้ในการพิมพ์สั่งงานบอทในช่องพิมพ์ข้อความแชทของ Google Chat (อย่าลืมพิมพ์ @ ชื่อบอทหน้าคำสั่งทุกครั้ง):

ความตั้งใจใช้งาน	ตัวอย่างการพิมพ์สั่งงาน	ผลลัพธ์ที่จะเกิดขึ้นบนระบบ Jira
เช็คสถานะบอท	@sekloso ping	บอทจะตอบกลับว่า 🟢 Pong! ระบบปกติทันที
สร้างงานหลัก (Epic)	@sekloso เพิ่มงาน ระบบสมัครสมาชิก	สร้างใบงานประเภท Epic บน Jira หัวข้อ "ระบบสมัครสมาชิก"
สร้างงานย่อย (Task)	@sekloso สร้าง job เขียน API ในงาน TES-10	สร้างใบงานย่อยประเภท Task ขึ้นไปอยู่ใต้ Epic งานชื่อ TES-10
เปลี่ยนสถานะการทำงาน	@sekloso ย้าย TES-5 ไปทำเสร็จแล้ว	เปลี่ยนสถานะตัว TES-5 เป็น Done (หรือสถานะอื่นๆ)
ตรวจสอบงานที่ค้าง	@sekloso ดูงานค้างทั้งหมด	บอทดึงรายการตัวงานที่ยังไม่เสร็จทั้งหมดส่งกลับมาแสดงในแชท

12. วิธีแก้ไขปัญหาที่พบบ่อย (Troubleshooting)

🔥 บอทไม่ตอบสนองหรือขึ้นเป็นสีแดงแจ้งเตือนหลังพิมพ์คุย

- **เช็ค API Key:** ปัญหาหลักมักเกิดจากลืมตรวจเช็ค API Key ของ Gemini และนารหัสของ Jira ที่ขึ้นต้นด้วย **AQ.** มาใส่สลับกัน (ตรวจและแก้ไขได้ที่ Vercel Environment Variables)
- **เช็ค URL Webhook:** ตรวจสอบว่าลิงก์ที่กรอกใน Google Cloud Console ได้กรอก **/api/webhook** ต่อท้ายครบถ้วนหรือไม่

🔥 งานบนหน้าบอร์ด Jira แก้ไขแล้ว แต่ข้อมูลไม่ยอมอัปเดตมาหน้า Dashboard

- **เช็ค Jira Webhook:** ตรวจสอบที่หน้าการตั้งค่า Jira Webhook ว่าสิทธิ์ของ Domain ใน URL ถูกต้องตรงกับ URL ปัจจุบัน และได้เลือกเหตุการณ์ Issue events ครบทั้ง Created, Updated และ Deleted แล้วหรือยัง

💡 การรีเซ็ตข้อมูลใหม่เพื่อรีเฟรชแคช (Force Reset Sync):

หากหน้าเว็บ Dashboard ไม่แสดงข้อมูลงานใดๆ เลยหลังติดตั้งเสร็จ คุณสามารถสั่งรีเฟรชข้อมูลทั้งหมดจาก Jira ใหม่ได้โดยการเปิดหน้าเบราว์เซอร์ไปที่ URL:

https://your-domain.vercel.app/api/cron/sync-tickets?secret=CRON_SECRET_ของคุณ